Fillcast: Queue-Aware Fill and Adverse-Selection Forecasting on a Deterministic C++ Hot Path

Standing Design Document

Ryan Hamby

2026-10-13

Contents

0. Document Control **2**

1. Abstract **3**

2. Goals and Non-Goals **3**

3. Problem Formulation **4**
3.1 Setting and notation . . . 4
3.2 Targets . . . 4
3.3 Why shadow-order labels, not real-order outcomes . . . 5
3.4 Latency awareness . . . 6
3.5 Assumptions and their status . . . 6

4. System Architecture **6**
4.1 Planes and the language boundary . . . 6
4.2 Data flow . . . 7
4.3 Components . . . 8
4.4 Determinism contract . . . 8
4.5 Engine changes required (from the current engine) . . . 8
4.6 Protocol layer, risk gate, and the closed order loop . . . 9
4.7 Integration adapters (off the latency path) . . . 10

5. ML Pipeline **11**
5.1 Stages . . . 11
5.2 Features . . . 11
5.3 Decision-epoch sampling . . . 11
5.4 Model ladder . . . 11
5.5 Leakage control and validation . . . 12
5.6 Calibration and uncertainty . . . 13
5.7 Quantization and train/serve parity . . . 14
5.8 Online adaptation, shadow mode, and promotion . . . 14
5.9 Monitoring . . . 14
5.10 Point-in-Time Assurance Suite . . . 14

6. Hot Path Design **15**
6.1 Threading . . . 15
6.2 Memory and allocation . . . 15
6.3 Latency budget . . . 16

6.4 Measurement protocol 16

7. High Availability and Fault Model **16**
7.1 Formal and simulation-based assurance 17

8. Telemetry **17**

9. Evaluation Plan **17**
9.1 Pre-registered hypotheses 17
9.2 Metrics 18
9.3 The L3 → L2 information-loss study 18
9.4 Ablations 18
9.5 Statistical protocol and negative results 19

10. Rust vs C++ Bake-off Protocol **19**

11. Phase C: Counterfactual Execution Policy **19**
11.1 A correction to the earlier plan 19
11.2 Scope 19
11.3 Validating A1 with real fills (optional, later) 19

12. Verification Strategy **20**

13. Data, Licensing, and Cost **20**

14. Risk Register **21**

15. Roadmap **21**
15.1 Definition of "80% in one week" 21
15.2 Beyond week one 22

16. Repository Layout and Reproducibility **22**

17. Claims Register **23**

18. References **23**

Appendix A: Glossary **24**

Appendix B: Building the Technical Report **24**

0. Document Control

Field	Value
Status	Draft v0.2, living document
Owner	Ryan Hamby
Substrate	Author's order book engine (C++20, price-time matching, SPSC ingestion)
Review cadence	Revised whenever an experiment confirms or kills an assumption
Final form	Technical report (Pandoc to LaTeX), see Appendix B

Evidence labels. Every quantitative statement in this document carries one label. A claim without a label is a bug in the document.

Label	Meaning
`[R]`	Reference: stated in the literature; citation in Section 18
`[H]`	Hypothesis: believed, not yet tested
`[T]`	Target: a budget or goal, not a result
`[M]`	Measured: backed by a captured artifact in `results/`, with hardware named

No number may be promoted to `[M]` without a dated capture. Negative results are published (Section 9.5).

Change log

Date	Version	Change
2026-10-13	0.1	Initial design after literature scan and adversarial review
2026-10-13	0.2	Added protocol layer (ITCH/MoldUDP64 in, OUCH out), pre-trade risk gate, point-in-time assurance suite, TLA+ and deterministic simulation testing, Kafka sink; roadmap re-sequenced

---

1. Abstract

Market makers are judged on realized execution, not on price-direction accuracy: capture per unit risk, adverse selection, and queue position. Fillcast is an end-to-end system that, for a hypothetical one-lot passive order at any price level, forecasts (i) the distribution of time-to-fill and (ii) the signed post-fill price move (markout), each with calibrated uncertainty, from exchange order-by-order (market-by-order, MBO) data. The prediction path runs on a deterministic, event-sourced, allocation-free C++ runtime built on the author's order book engine.

The design makes eight commitments that distinguish it from a typical quant-developer project.

1. **Counterfactual shadow-order labels.** Training labels come from replaying the exact order stream with non-intrusive hypothetical orders. Because shadow orders never cancel, the labels contain no informative censoring (Section 3.3).
2. **Latency-aware formulation.** The order lands δ after the decision, at a queue position unknown at decision time. δ is a first-class parameter and all results are reported as functions of it (Section 3.4).
3. **Leakage by construction.** One sequenced journal is the single source of truth. A knowability clock governs every feature. Training and serving share one integer-only code path, verified bit-exact (Sections 4.4, 5.5).
4. **Measured information loss.** MBO ground truth lets us degrade the feed to aggregate L2, re-infer queue position, and quantify what backtests built on L2 get wrong (Section 9.3).
5. **Operational rigor.** Deterministic replicas with state-hash divergence detection, model hot-swap with guardrails, and a fault-injection test per HA claim (Sections 6, 7).
6. **Pre-registered evaluation.** Hypotheses, metrics, and thresholds are logged before experiments run. Failures are published (Section 9).
7. **Real wire protocols and a closed order loop.** Market data enters as ITCH 5.0 over MoldUDP64; orders leave as OUCH through a pre-trade risk gate to a mock exchange built on the engine; execution reports return through the same journal (Sections 4.6, 7).
8. **Verifiable point-in-time correctness.** Leakage freedom is stated as a formal property (prefix invariance), tested by mutation, audited per dataset row, and tied to a hash-chained provenance record that a `verify` command checks (Section 5.10).

---

2. Goals and Non-Goals

#	Goal	Done when
G1	Calibrated multi-horizon fill probabilities for shadow orders	Calibration passes pre-registered thresholds out of sample (H2)
G2	Calibrated post-fill markout forecasts with interval coverage	Coverage within tolerance by regime (H6)
G3	A decision layer whose value is expressed as net edge per post	Lower CI bound beats the heuristic baseline (H1)
G4	Bit-reproducible train/serve parity	Golden-vector test passes in CI (H5)
G5	Deterministic hot path with measured tail latency	Budgets in Section 6.3 met or honestly missed (H9)
G6	Fault-tolerant operation under injected faults	Chaos suite passes (H8)
G7	A reproducible research artifact	Anyone can rerun `make reproduce` from manifests
G8	Protocol-faithful ingest and order entry with a closed, replayable loop	Decoders/encoders pass conformance, fuzz, and fault-injection tests (H11, H13)
G9	Point-in-time correctness a reviewer can verify rather than trust	PIT suite passes and `verify` accepts the provenance chain (H12)

#	Non-goal	Reason
N1	Claim alpha, Sharpe, or P&L	Replay with shadow orders cannot support it
N2	Claim tick-to-trade latency	No NIC, kernel-bypass, or wire path is involved. The metric is *decision latency from an in-memory message*
N3	Trade live	Out of scope; see Section 11.3 for validation options
N4	Beat deep models on benchmark leaderboards	Evaluation is in decision terms, not F1
N5	Multi-region or multi-venue HA	Single-AZ, two replicas
N6	Claim exchange certification or production protocol conformance	Codecs are tested against published specifications and a mock exchange only; the risk gate protects a simulated loop
N7	Put Kafka, or any broker, on the latency path	The sink is a sidecar fed from the journal (Section 4.7)

3. Problem Formulation

3.1 Setting and notation

The exchange emits a sequenced stream of order-level events e_n, $n = 1, 2, \ldots$ (add, cancel, modify, trade). Each carries an exchange time t_n^{ev} and a capture (receive) time t_n^{rx}. The book state after event n is B_n.

A **decision epoch** is an event n that touches the top L levels. The decision is made at

$$t_n = t_n^{\text{rx}} + \delta_{\text{proc}},$$

using only events with $t^{\text{rx}} \le t_n$ (the *knowability clock*). The order reaches the matching engine at

$$a_n = t_n + \delta_{\text{net}}, \qquad \delta \equiv \delta_{\text{proc}} + \delta_{\text{net}}.$$

A **shadow order** o is a one-lot passive order with side $s \in \{+1\ (\text{bid}), -1\ (\text{ask})\}$ and level offset ℓ from the touch, inserted at the back of its queue at the first event with $t^{\text{ev}} \ge a_n$. Let Q_o be the lots resting ahead of it at insertion. Because $a_n > t_n$, Q_o is **not observable at decision time** and must be forecast.

3.2 Targets

Time to fill. Under FIFO, o fills once every lot ahead has been removed by executions or cancels and a subsequent execution reaches o. Let T_o be that time, measured from insertion. Labels are administratively censored at $\tau_{\max}$ or session end:

$$Y_o = \min(T_o, \tau_{\max}), \qquad \Delta_o = \mathbb{1}[T_o \le \tau_{\max}].$$

Time is discretized on a log-spaced grid $0 = b_0 < b_1 < \cdots < b_K = \tau_{\max}$ (for example 1 ms to 60 s). The **discrete-time hazard** is

$$h_k(x) = \Pr\left(T_o \in (b_{k-1}, b_k] \mid T_o > b_{k-1},\, x\right),$$

with survival and cumulative fill probability

$$S(b_k \mid x) = \prod_{j \le k} (1 - h_j(x)), \qquad F(b_k \mid x) = 1 - S(b_k \mid x).$$

The training loss is the person-period negative log-likelihood, a sum of Bernoulli terms over the periods each order is at risk:

$$\mathcal{L}(\theta) = -\sum_o \sum_{k \le k_o} \Big[d_{o,k} \log h_k(x_o; \theta) + (1 - d_{o,k}) \log\left(1 - h_k(x_o; \theta)\right) \Big],$$

where $d_{o,k} = 1$ only in the period where o fills. Hazard heads are monotone-consistent by construction, which avoids crossing cumulative curves.

Markout. For a fill at price p_o and time T_o, with horizon h:

$$m_o(h) = s\left(\mathrm{mid}(a_o + T_o + h) - p_o\right), \qquad m_o^{\mathrm{net}}(h) = m_o(h) - f,$$

where f is the per-lot fee (negative for a rebate). Markout is a regression target on *filled* orders only, and conditioning on a fill matters: fills occur disproportionately when the market moves through the level (winner's curse), so $\mathbb{E}[m \mid \text{fill}] < \mathbb{E}[m]$.

Edge and decision.

$$\mathcal{E}_{\tau,h}(x) = F(\tau \mid x) \Big(\mathbb{E}\big[m_o(h) \mid T_o \le \tau,\, x\big] - f \Big), \qquad \text{post} \iff \underline{\mathcal{E}}_{\tau,h}(x) > c,$$

where $\underline{\mathcal{E}}$ is a lower confidence bound (Section 5.6) and c is an opportunity-cost threshold. All results report net edge per post as a function of (f, δ, c) rather than one headline number.

3.3 Why shadow-order labels, not real-order outcomes

An earlier sketch trained on the fates of real resting orders. That is the wrong target for two reasons.

1. **Informative competing risk.** Real participants cancel because they have information. Treating their cancels as censoring biases the fill hazard, and under dependent competing risks the cancel-free marginal fill time is not identifiable from the data alone (Tsiatis non-identifiability, `[R]`). Cause-specific modeling fixes the estimand but not the question: *we* will never cancel on the same information.
2. **We want the fate of an order we control.** A shadow order follows a policy we define (hold to $\tau_{\max}$), so its only censoring is administrative.

Under the assumptions in Section 3.5, the label for any shadow order is exactly computable by replay, with no statistical estimation. Consequences:

- No inverse-probability-of-censoring weighting is needed; IBS and Brier are computed directly.
- Labels exist for every level, side, and epoch simultaneously (full-information feedback, relevant to Section 11).
- **Labeler validation is an exact-match test**, not a fit statistic. Real orders that rest unmodified until filled must be reproduced to the same event by a shadow order inserted behind the same queue position.

3.4 Latency awareness

Because Q_o at insertion differs from Q at decision, a model that ignores δ is optimistic. We treat δ as an explicit grid, $\delta \in \{0,\ 50\,\mu\text{s},\ 500\,\mu\text{s},\ 5\,\text{ms}\}$ [H], report every metric at each value, and test directly whether ignoring δ inflates edge (H3). The model's input x is computed at t_n; the label's queue position is realized at a_n.

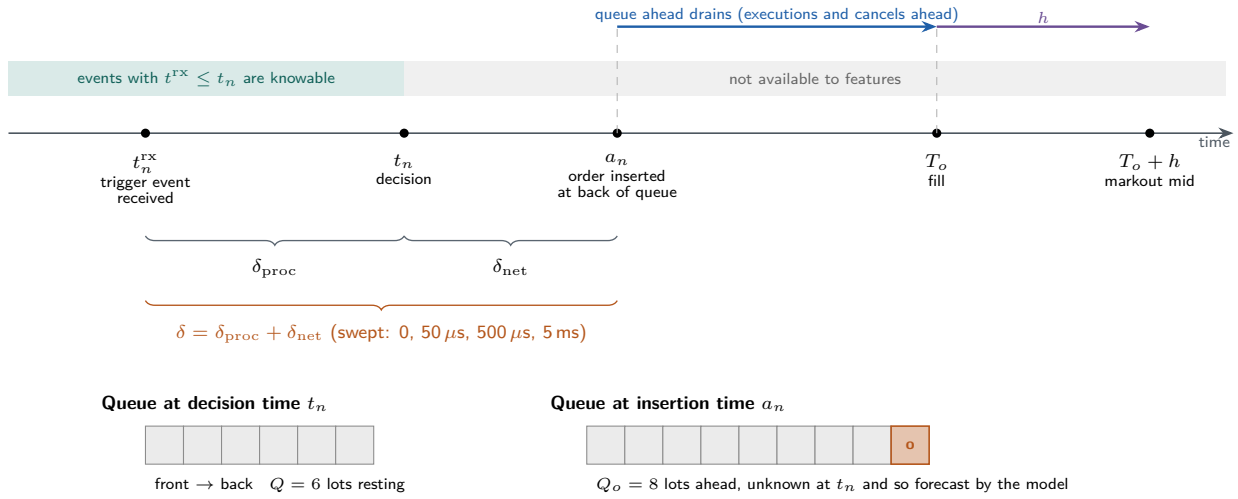


Figure 1: Decision time versus insertion time. Features may use only events received by t_n; the order joins the queue at a_n, where its queue position is unknown at decision time and must be forecast.

3.5 Assumptions and their status

ID	Assumption	Risk if violated	Handling
A1	**No impact:** a one-lot shadow order does not change others' behavior	Fill rates biased in unknown direction	Queue-jump stress test; restrict to deep queues; report as sensitivity band; validation options in 11.3
A2	**FIFO priority** at the venue and product	Wrong fills	Choose FIFO products only; assert venue rules in the engine
A3	**Feed completeness**	Corrupt queue state	Sequence-gap detection; reconcile reconstructed book to the provider's book after every event
A4	**Knowability:** features use only $t^{\text{rx}} \le t_n$	Silent leakage	Canary tests (Section 5.5)
A5	**Local stationarity** within validation folds	Miscalibration	Rolling monitors, regime-conditional reporting
A6	**Priority rules** (modify-loses-priority on size increase, keeps on decrease) are implemented as in the venue	Wrong queue positions	Differential tests against venue-published book

---

4. System Architecture

4.1 Planes and the language boundary

The key architectural decision is that **the Rust/C++ boundary is a documented binary format and a shared-memory ring, not an FFI**. This keeps each language idiomatic, makes either side replaceable,

and is what makes the Rust-vs-C++ bake-off (Section 10) honest: both implementations must emit a byte-identical journal.

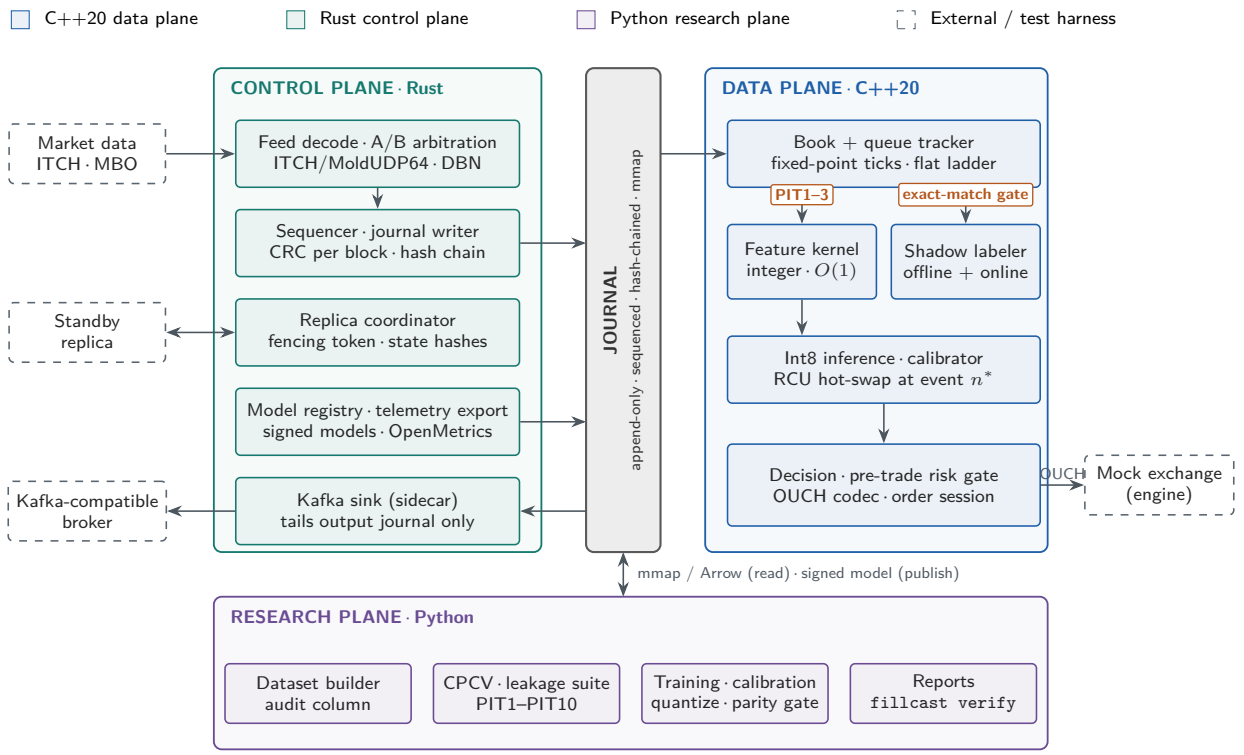


Figure 2: System architecture. The Rust control plane and C++ data plane meet at a documented binary journal and shared-memory rings, not an FFI. The Python research plane reads the journal read-only and publishes signed models through the registry.

4.2 Data flow

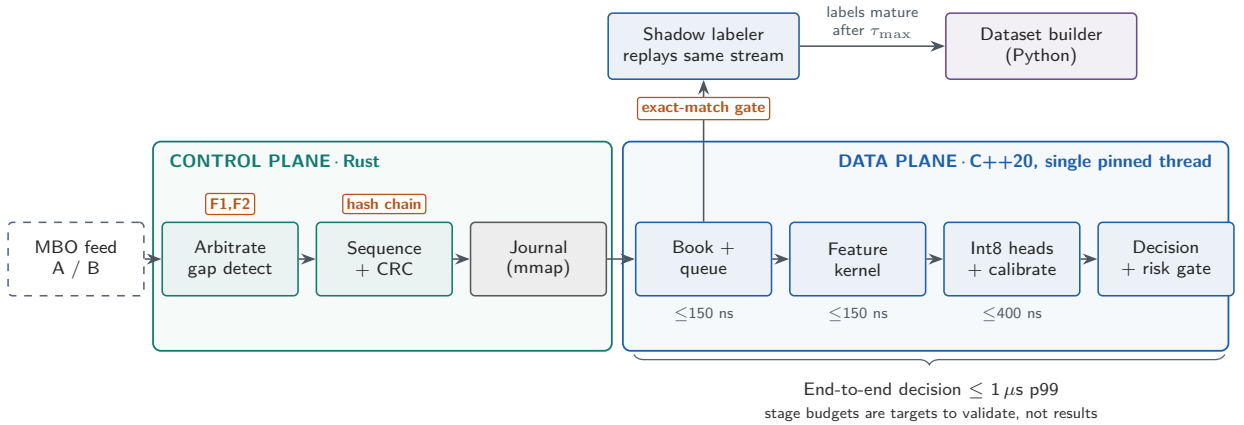


Figure 3: Live event path and stage budgets. Budgets are targets to validate (labeled [T] in the text), not measurements. The shadow labeler replays the same stream and feeds the offline dataset.

4.3 Components

Component	Plane	Language	State	Notes
Feed decoder, A/B arbiter	Control	Rust	Per-line seq cursors	Gap detection, dedupe, retransmit/snapshot recovery (simulated by fault injection on replay)
Sequencer + journal writer	Control	Rust	Segment catalog	Fixed-width 64-byte records, CRC per block [H]
Book + queue tracker	Data	C++20	Fixed-point ticks, flat price ladder, pooled order nodes	Extends the existing engine; `std::map` removed
Shadow-order labeler	Data	C++20	Sparse shadow queue state	Never mutates the real book
Feature kernel	Data	C++20	Integer EWMAs, counters	Same object linked into Python via a thin binding for training
Inference	Data	C++20	Immutable model blob	Integer-only; swapped via RCU pointer
Replica coordinator	Control	Rust	Lease/fencing token, state hashes	Compares hashes every 2^{16} events [H]
Telemetry exporter	Control	Rust	None	Reads hot-path counters; OpenMetrics
Model registry	Control	Rust/Python	Versioned, signed blobs	Promotion gates in Section 5.8
Research tooling	Research	Python	None	CPCV, leak suite, reports
ITCH / MoldUDP64 handler	Control	Rust (C++ twin later)	Per-session seq cursors	Gap detect, retransmit request, dedupe; feeds the same journal as the DBN decoder
OUCH codec + order session	Data	C++20	Session state machine	Login, sequence numbers, heartbeat, recovery
Pre-trade risk gate	Data	C++20	Limit counters, kill-switch flag	Fail-closed; deterministic; Section 4.6
Mock exchange	Test harness	C++20	Engine instance	Built on the order book engine; returns acks, fills, rejects
Kafka sink	Control	Rust sidecar	Output offset cursor	Tails the output journal; never on the latency path (Section 4.7)

4.4 Determinism contract

The data plane is a pure fold over the journal: $(\text{state}_n, \text{output}_n) = f(\text{state}_{n-1}, e_n)$. To make that true in practice:

1. **No wall clocks** in logic; only event times. Clocks are used for telemetry only.
2. **Integer arithmetic in the book, features, and inference.** Prices are tick integers; features are fixed-point (Q-format) integers; inference is int8 with int32 accumulators and integer requantization; the sigmoid is a lookup table.
3. **No `libm` and no floating point in the hot path.** Where floats remain (offline), `-ffp-contract=off` is mandatory.
4. **No unordered-container iteration** in anything that affects state. No dependence on pointer values or thread scheduling.
5. **Side threads communicate by sequenced messages**, applied at event boundaries. A model swap or online update takes effect at a named event number n^* on every replica.
6. **Guarantee scope:** bit-identical output for a given (binary hash, ISA). Cross-ISA parity (x86-64 vs ARM64) is tested in CI and expected to hold because the path is integer-only [H].

If an automated cross-compiler/cross-ISA hash test fails, the failure is a defect, not an accepted limitation.

4.5 Engine changes required (from the current engine)

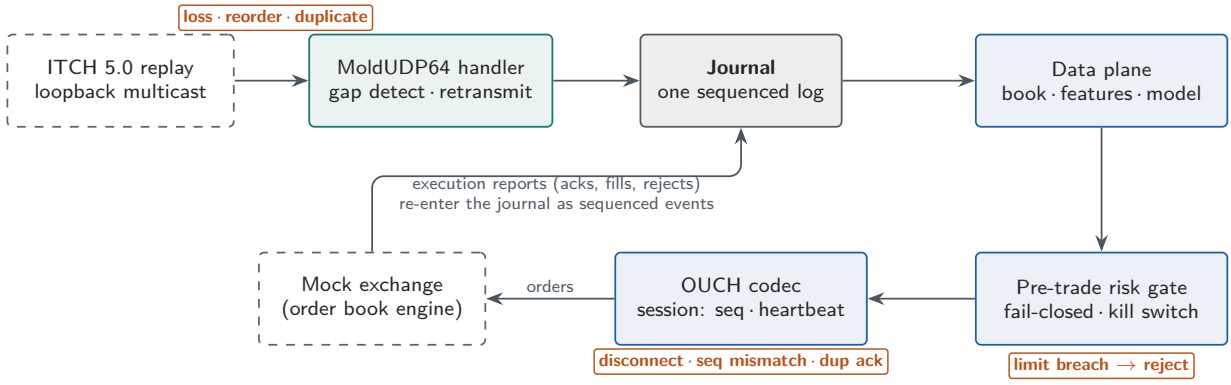
Current	Required	Why
`double` prices	Fixed-point tick integers	Determinism, exact equality
`std::map` price levels (measured as the dominant cost in the existing flame graph)	Flat ladder indexed by tick offset with a sliding window	Removes red-black rebalancing, tail latency
Add + match only	Add, cancel, modify (priority rules), trade-reporting, replace	Real MBO semantics
Synthetic random orders	Journal replay + shadow orders	The actual use case
Mutex vs SPSC comparison	Retained as a regression benchmark	Already a good artifact

4.6 Protocol layer, risk gate, and the closed order loop

Ingest and order entry use real wire formats, so they are exercised rather than stubbed.

Direction	Protocol	Role	Scope here
Market data in	Nasdaq TotalView-ITCH 5.0 over MoldUDP64	Sequenced UDP multicast; gap detection; retransmission requests	Replayed over loopback multicast with injected loss, reordering, duplication
Market data in	Vendor normalized MBO (DBN)	Primary research dataset (CME futures)	Decoder only; not a wire protocol
Order entry out	OUCH over a SoupBinTCP-style session	Enter, replace, cancel; acks, fills, rejects	Encoder/decoder and session state machine (login, sequence numbers, heartbeat, recovery) against the mock exchange
Internal	Journal records	Binary, fixed-width	Section 4.3

Protocol versions and session details are taken from the published specifications and must be re-checked against the current revisions before any conformance statement is made.



Closed loop: every input to the data plane is a journal record, so the whole loop replays bit-identically and fault schedules can be seeded.

Figure 4: Protocol layer and closed order loop. Execution reports re-enter the journal as sequenced events, so the loop replays deterministically. Orange tags mark injected faults and gates.

Execution reports re-enter the **same journal** as sequenced events, so the entire closed loop is deterministic and replayable.

Pre-trade risk gate. Controls sit on the output path and fail closed.

Control	Check	On breach
Order size	Max lots per order	Reject
Notional	Max notional per order and in aggregate	Reject
Price collar	Order price within a band around a reference price (fat-finger)	Reject
Rate	Max messages per second per instrument and overall	Throttle, then reject
Position	Max net position and open-order count	Reject
Duplicate guard	Same client order id or identical order within a window	Reject
Stale data	Market data older than a threshold [T]	Block entry
Kill switch	Manual, or automatic on divergence, calibration, or stale-data alarm	Cancel all; block entry until reset

Each control has a must-block test, and a property test checks that no generated sequence of orders passing the gate exceeds any limit. The gate is modeled on the kind of pre-trade controls market-access rules require (for example SEC Rule 15c3-5, [R]); the project makes no compliance claim, and it never trades, since the gate guards a simulated loop.

4.7 Integration adapters (off the latency path)

A Kafka-compatible sink publishes the sequenced output stream and telemetry for downstream consumers (dashboards, risk monitors, a research data lake), the way such systems are typically wired into a firm's middleware.

Aspect	Design
Source	A Rust sidecar tails the **output journal**; it never touches data-plane memory
Delivery	Idempotent publish keyed by output sequence number; consumers can detect gaps and duplicates by sequence
Isolation	Broker slowness or outage cannot back-pressure the data plane, because the journal decouples them; the sink catches up from its stored offset
Topics	Per instrument and stream: predictions, decisions, execution reports, telemetry
Local setup	Single containerized broker; no managed service
Evidence	H14: hot-path p99 and p99.9 with the sink on versus off; broker killed for a fixed period, then zero gaps and zero duplicates after recovery (fault F11)

The journal already provides the log semantics (append-only, sequenced, replayable); the sink exists to show integration, not to replace it.

5. ML Pipeline

5.1 Stages

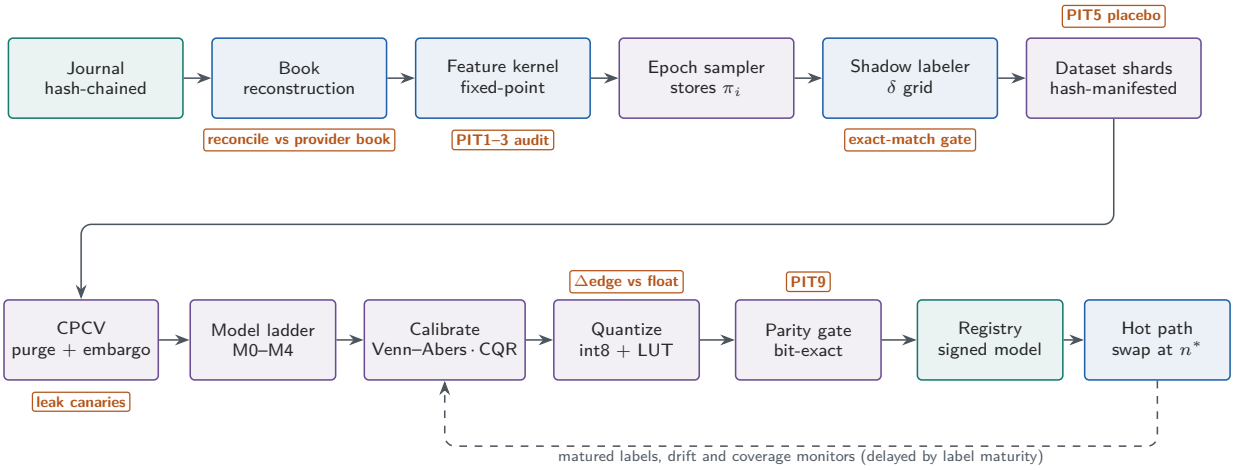


Figure 5: Pipeline from journal to hot-swapped model. Orange tags mark gates that must pass before the next stage (Sections 5.5, 5.7, 5.10).

5.2 Features

All features are computed incrementally in $O(1)$ per event from the kernel's state, in fixed-point.

Group	Examples	Rationale
Queue state	Lots ahead/behind at each of top L levels; level age; last-trade-at-level age	Direct determinants of depletion
Order-flow imbalance	Multi-level OFI over several windows (Cont, Kukanov, Stoikov [R])	Short-horizon pressure
Flow intensity	EWMA of signed and unsigned trade rate, cancel rate, add rate at several time constants	Hawkes-like self-excitation without fitting a point process
Book shape	Spread, depth ratios, imbalance, slope	Resistance to being run over
Volatility / time	Event-time and clock-time realized volatility proxies; time-of-day buckets; time since last event	Regimes, session effects
Cross-instrument (later)	Correlated contract's imbalance and returns	Lead-lag

The feature set is decided by ablation (Section 9.4), not by taste. Fixed-point error against a float reference is measured and budgeted.

5.3 Decision-epoch sampling

Labeling every epoch at every level is wasteful and highly autocorrelated. The sampler draws epochs stratified by level and side and records the inclusion probability π_i with each row, so evaluation can reweight to a time-uniform distribution when needed. Effective sample size is reported using day-block resampling, since overlapping horizons make rows non-independent.

5.4 Model ladder

Every model is trained and evaluated under identical splits. Higher rungs must earn their complexity in decision terms.

Rung	Model	Purpose	Deployed form
M0	Heuristic: $P(\text{fill}) = g(Q/\hat{\lambda}_{\text{depletion}})$	The baseline a practitioner would beat first	Closed form
M1	Discrete-time logistic hazard	Interpretable, strong on small data	Integer dot products
M2	Gradient-boosted trees on person-period data	Strong tabular baseline	Compiled integer-threshold code
M3	Small MLP, multi-head (hazard bins × markout quantiles)	Primary candidate; shares representation across heads	Int8 SIMD
M4	Queue-reactive / Hawkes-style structural model [R]	Literature baseline	Offline only

Recent evidence that simple models match deep ones on LOB tasks (the TLOB paper's MLP result [R]) is why M0 and M1 are mandatory comparators, not strawmen.

5.5 Leakage control and validation

Threat model. Each row lists a way the pipeline could cheat and the test that must catch it.

Leak source	Example	Gate
Wrong clock	Using exchange time where the system could only know capture time	Knowability test: recompute features with events truncated at t_n^{rx}; must match bit-for-bit
Label window overlap	Train rows whose horizon spans a test boundary	Purge by label interval; embargo after each test block
Global statistics	Normalizing with all-data mean/variance	Fit scalers only inside training folds; assert in code
Hyperparameter peeking	Selecting on the test fold	Nested validation; sealed final holdout opened once
Multiple testing	Many configs, report the best	Trial registry; PBO and deflated metrics
Session artifacts	Contract roll, auction, halts	Explicit exclusion rules; unit-tested calendars
Train/serve skew	Different feature code offline vs live	One feature kernel; golden-vector parity test

Leak canaries (must-fail tests). CI plants deliberate leaks and the suite must flag every one:

1. A feature that peeks one event into the future.
2. A feature using the label's own horizon-end mid-price.
3. A scaler fit on test data.
4. Shuffled labels must score at chance within a confidence interval.
5. A time-reversed dataset must degrade performance.

Combinatorial purged cross-validation (CPCV). Split the trading days into N contiguous groups and hold out k as test. There are $\binom{N}{k}$ splits, and each group appears in test sets across φ distinct backtest paths:

$$\varphi(N, k) = \frac{k}{N}\binom{N}{k}.$$

For $N = 6$, $k = 2$: 15 splits and 5 paths. Purging removes any training row whose label interval $[a_o,\ a_o + \tau_{\max} + h_{\max}]$ overlaps a test interval widened by the maximum feature lookback; an embargo of at least one trading day follows each test block. Reporting is over paths, giving a distribution rather than a point estimate [R].

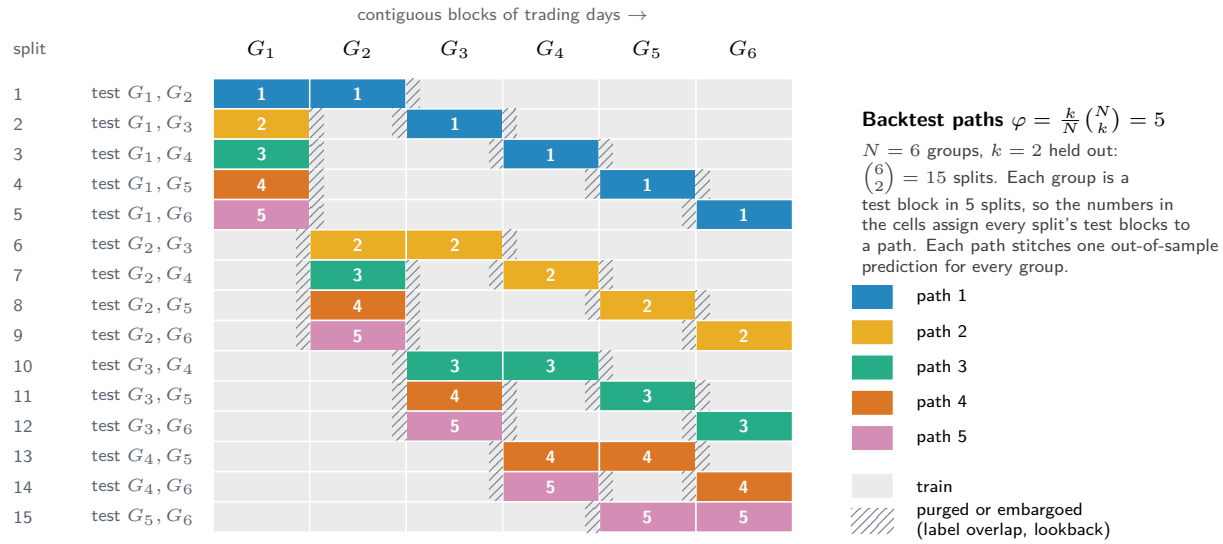


Figure 6: Combinatorial purged cross-validation for $N = 6$, $k = 2$: 15 splits recombined into 5 backtest paths. Hatched cells are purged or embargoed training days next to each test block.

Selection bias control. The number of trials N_{trials} is logged in the registry. Configuration selection is evaluated with the probability of backtest overfitting (via combinatorially symmetric cross-validation, [R]). For any decision-layer return series, the deflated Sharpe ratio uses the expected maximum under N_{trials} independent trials:

$$\widehat{SR}_0^* = \sqrt{\text{Var}[\{\widehat{SR}_n\}]}\left[(1-\gamma)\,\Phi^{-1}(1-\tfrac{1}{N_{\text{trials}}}) + \gamma\,\Phi^{-1}(1-\tfrac{1}{N_{\text{trials}}\,e})\right],$$

with γ the Euler–Mascheroni constant. This is a hygiene check on selection, not a performance claim.

5.6 Calibration and uncertainty

- **Fill probability.** Post-hoc isotonic or Venn–Abers calibration [R] on a held-out calibration block per horizon. Reliability diagrams, Brier score, integrated Brier score

$$\text{IBS} = \frac{1}{\tau_{\max}} \int_0^{\tau_{\max}} \text{BS}(t)\, dt,$$

  and adaptive-bin ECE. Since labels are exact up to $\tau_{\max}$, no IPCW is required.
- **Markout.** Conformalized quantile regression (CQR): score $E_i = \max\{\hat{q}_{\text{lo}}(x_i) - y_i,\ y_i - \hat{q}_{\text{hi}}(x_i)\}$. Under distribution shift, adaptive conformal inference [R] updates the miscoverage level online:

$$\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{err}_t), \qquad \text{err}_t = \mathbb{1}[y_t \notin C_t].$$

  Labels mature only after $\tau_{\max} + h$, so the update runs **with delayed feedback**; the delay is part of the design, not an afterthought.
- **Reporting.** Marginal coverage hides failures. Coverage is reported conditionally by volatility tercile, time of day, and scheduled-event days.
- **Decision.** The rule uses the lower bound $\underline{\mathcal{E}}$ so that uncertainty changes behavior, which is the point of computing it.

5.7 Quantization and train/serve parity

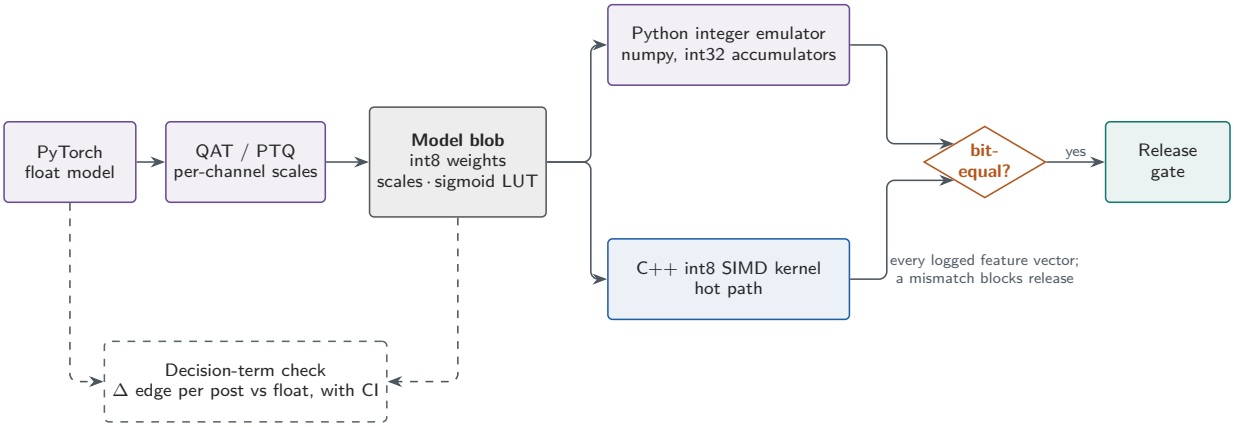


Figure 7: Train/serve parity. Integer emulation in Python and the C++ kernel must agree bit for bit on every logged feature vector; the accuracy cost of quantization is measured separately in decision terms.

Weights are int8 per-channel, activations int8 per-tensor, accumulation in int32, requantization by integer multiplier and shift. The activation nonlinearity is a lookup table. The Python emulator and the C++ kernel must agree on every logged feature vector. Separately, the quantization cost is measured **in decision terms**: change in net edge per post versus the float model, with CI.

5.8 Online adaptation, shadow mode, and promotion

Online learning in a trading path is a hazard, so the design limits it.

- **Only a small head adapts online:** the calibrator and, optionally, the last layer. The backbone is retrained offline.
- **Delayed labels:** an update uses a label only once matured, applied at a named event number so every replica stays identical.
- **Shadow mode:** a challenger runs on the same stream and writes predictions without influencing the decision output.
- **Promotion gates (all required):**
  1. Calibration non-inferior to champion on matured labels.
  2. Lower CI bound on edge improvement above zero over a fixed window.
  3. Latency budget met.
  4. No active drift alarm.
- **Guardrails:** bounded update magnitude, output sanity bounds, a kill switch, and automatic rollback to the last-good model.

5.9 Monitoring

Rolling Brier and ECE on matured labels, feature PSI against the training reference (integer-binned), interval coverage, fill-rate drift, label-maturity lag, and per-model-version tagging of every output.

5.10 Point-in-Time Assurance Suite

The aim is to make "no leakage" something a reviewer can check rather than take on trust.

Formal property. Let ϕ map a journal prefix to the feature vector at each epoch, and let $J_{1:n}$ be the first n records. Features must satisfy *prefix invariance*: for every epoch $m \le n$ and every possible suffix S,

$$\phi(J_{1:n} \,\|\, S)_m = \phi(J_{1:n})_m.$$

The test applies random mutation, deletion, and truncation to the suffix and requires bit-identical features for all earlier epochs. Labels are the only component allowed to read forward, and they live in an isolated module with an explicit allowlist.

Provenance chain. Journal segments are hash-chained,

$$H_i = \text{SHA-256}(H_{i-1} \,\|\, \text{segment}_i),$$

a dataset manifest records the chain range and the code hash, a model card records the dataset hash, training code hash, and seeds, and every result manifest records the model hash. `fillcast verify` recomputes the chain and CI rejects any result whose chain does not verify.

ID	Control	Mechanism	Catches	Gate
PIT1	Prefix invariance	Random suffix mutation, deletion, truncation	Any use of future events in features	100% of sampled epochs bit-identical
PIT2	Streaming-only feature API	Kernel exposes only `on_event`; no random access to the journal; header-dependency lint forbids feature code from including label code	Structural lookahead	CI lint passes
PIT3	Row audit column	Each dataset row stores the highest event sequence its features consumed	Silent lookahead in the dataset builder	Builder asserts value $\leq$ epoch sequence
PIT4	Independent reference kernel	Slow integer Python implementation written from the specification	Implementation and specification errors	Bit-exact agreement with C++
PIT5	Placebo labels	Labels taken from a shifted time or a different day	Leakage into the label or split logic	Scores at chance
PIT6	Bitemporal reference data	Tick size, multiplier, fees, calendar, and roll map carry an effective date and a recorded-as-of date, and are pinned by hash	Retroactively revised reference data	Version used at any epoch was recorded at or before that epoch
PIT7	Hash-chained lineage	The chain above	Edited or untraceable data	`verify` passes
PIT8	Leak canaries	Section 5.5	Detector blind spots	100% flagged
PIT9	Train/serve parity	Section 5.7	Skew between offline and live code	Bit-exact
PIT10	Leakage report	Generated each release: every test, its outcome, and the manifest hashes	A reviewer unable to audit	Report committed with the release tag

---

6. Hot Path Design

6.1 Threading

Thread	Role	Notes
T0 (pinned)	Event loop: book, features, labeler, inference, decision	Single writer; all state changes happen here
T1	Telemetry flush	Reads relaxed counters; never touches hot state
T2	Model-update stager	Prepares blobs; publishes by sequenced message

6.2 Memory and allocation

Preallocated arenas; zero `malloc` after initialization, enforced in CI by an allocation trap that aborts on any call during replay. Flat price ladder indexed by tick offset; intrusive FIFO per level over a pooled node array; open-addressing order-id table. Cache-line padding on cross-thread counters.

6.3 Latency budget

These are **targets [T]** to be validated or refuted; none is a result.

Stage	Budget [T]
Event apply (book + queue)	$\leq$ 150 ns median
Incremental feature update	$\leq$ 150 ns median
Int8 multi-head inference	$\leq$ 400 ns median
End-to-end decision	$\leq$ 1 µs p99
Tail	p99.9 reported with max; no claim before measurement

Model size is deliberately small (roughly 40 inputs, two hidden layers of 64, four hazard heads plus markout heads), so the arithmetic is on the order of 10^4 multiply-accumulates per decision [H].

6.4 Measurement protocol

- **Open-loop replay with intended send times.** A closed-loop benchmark that waits for the previous response understates tail latency (coordinated omission). Latency is measured from *intended* arrival.
- HDR histograms per stage; warm-up discarded; report p50, p99, p99.9, max, and sample count.
- Three rates: real-time, accelerated, and unthrottled burst.
- Isolated cores and fixed frequency; one bare-metal session for hardware counters (virtualized instances do not expose them, as observed on the existing benchmark host).
- Every figure comes from a captured run naming the hardware. Laptop numbers are labeled informal.

---

7. High Availability and Fault Model

"HA" is claimed only for faults with a passing injection test.

ID	Fault	Detection	Mitigation	Test
F1	Packet loss (MoldUDP64 sequence gap or one A/B line)	Sequence gap	Arbitrate from the other line; request retransmission	Inject drops at rate p on loopback multicast; zero lost events
F2	Duplicates / reordering	Sequence numbers	Dedupe and reorder buffer with bounded window	Inject both; output identical to clean run
F3	Primary crash	Lease expiry / heartbeat	Hot standby promotes under a fencing token	`kill -9` mid-stream; measure time to first correct output; no duplicate/lost outputs
F4	Silent divergence	State hash every 2^{16} events	Halt output, alert, resume from last agreed checkpoint	Flip a bit in one replica; detected within one interval
F5	Split brain	Fencing token monotonicity	Stale primary's outputs rejected	Partition the pair; only one stream accepted
F6	Bad or poisoned model	Signature, shadow gate, sanity bounds	Reject / auto-rollback	Inject a poisoned update; guardrail trips
F7	Slow consumer	Ring occupancy	Journal is the source of truth; consumer lags but never drops	Throttle consumer; no loss, lag alarmed
F8	Storage corruption	CRC per block	Quarantine segment; refetch	Corrupt a block; detected and isolated
F9	Order-entry session fault (disconnect, sequence mismatch, duplicate ack)	Session state machine	Re-login, resynchronize sequence numbers, reconcile open orders against mock-exchange state	Inject each; open-order state converges with no duplicate orders

ID	Fault	Detection	Mitigation	Test
F10	Risk limit breach	Pre-trade gate	Reject, count, alert; repeated breaches trip the kill switch	Property test: no gate-passing sequence exceeds a limit
F11	Broker outage (Kafka sink)	Producer errors, offset lag	Sink retries from the output journal; data plane unaffected	Kill the broker for a fixed period; hot-path latency unchanged within noise; zero gaps or duplicates after recovery

Scope. Two replicas, one availability zone. Recovery-time numbers are produced by the F3 test; no target is asserted in advance. If the determinism contract (Section 4.4) cannot be met, the word "HA" is replaced by "deterministic replay with checkpoint recovery" in all documents.

7.1 Formal and simulation-based assurance

TLA+ specification. The replica protocol (lease, fencing token, promotion, state-hash checkpoint) is specified in TLA+ and model-checked with TLC over a small bounded configuration. Invariants:

- *SingleWriter:* at most one primary can emit in any fencing epoch.
- *GaplessOutput:* accepted output sequence numbers are contiguous.
- *SafePromotion:* a promoted replica's state equals the last agreed checkpoint plus the journal suffix.

Model checking finds design errors; it does not prove the implementation correct. The spec's scope and its bounds are stated in the repository.

Deterministic simulation testing. Because the data plane is a pure fold and the control plane's sources of nondeterminism (time, network, scheduling) can be injected, the replica pair runs inside a single-threaded simulated environment driven by a seeded fault schedule (drops, delays, crashes, partitions, session faults). Any failing seed reproduces the failure exactly. A CI subset of seeds runs on every change and a larger set runs nightly; the report lists seeds run, failures found, and fixes.

---

8. Telemetry

Tier	Signal	Collection	Constraint
Latency	Per-stage HDR histograms	Cycle counter on hot path, calibrated to ns; flushed by T1	Overhead measured as an ablation with telemetry on/off
Throughput	Events/s, decisions/s, queue depths	Per-thread cache-aligned counters	No locks, no formatting on hot path
Model health	Rolling Brier/ECE, coverage, feature PSI, fill-rate drift	Computed on matured labels	Delayed by label maturity; displays the delay
System	Replica lag, gap counts, failover events, hash agreement	Control plane	Alert thresholds in repo
Trace	Sampled event-to-decision traces ($1/2^k$)	Ring buffer	Disabled in latency runs unless stated

Exported as OpenMetrics, visualized in committed dashboard definitions, with screenshots stored as result artifacts.

---

9. Evaluation Plan

9.1 Pre-registered hypotheses

Thresholds marked TBD are fixed and logged (with `pow log`) before the corresponding experiment runs. They are not tuned afterward.

ID	Hypothesis	Primary metric	Baseline	Pass condition
H1	Learned hazard beats the queue-depletion heuristic	Net edge per post, CPCV path distribution	M0	Lower 95% CI of difference > 0
H2	Fill probabilities are calibrated out of sample	ECE, IBS	M0, M1	ECE $\leq \varepsilon_1$ (TBD) on every path
H3	Ignoring δ inflates edge	Edge(δ=0) minus edge(δ)	n/a	Difference significant at each $\delta > 0$
H4	L2-inferred queue position loses material accuracy	Position error; calibration and edge degradation	MBO truth	Quantified with bands under front/random/back rules
H5	Integer pipeline is bit-exact and cheap in accuracy	Bit equality; edge delta vs float	Float model	Bit-exact; edge delta $\leq \varepsilon_2$ (TBD)
H6	Markout intervals achieve nominal coverage, including under shift	Conditional coverage by regime	Static split conformal	Within tolerance ε_3 (TBD)
H7	Leak suite is effective	Canary detection rate	n/a	100% of planted leaks flagged
H8	HA claims hold under injection	Divergence, loss, failover time	n/a	Zero divergence/loss for F1–F8
H9	Hot path meets its budget	p50/p99/p99.9 per stage	Section 6.3	Reported either way
H10	Rust vs C++ ingest	Throughput, tail, defect count	n/a	Reported without a pre-judged winner
H11	Codecs conform to the published specifications	Round-trip, differential vs an independent reference parser, fuzzing	Reference parser	Zero mismatches on the corpus; fuzzing clean
H12	The PIT suite catches leakage and passes on a clean pipeline	PIT1–PIT10 outcomes	Planted leaks	Zero false negatives on planted leaks; zero violations on the clean run
H13	The risk gate holds	Property tests; breach injection	n/a	No limit exceeded by any generated sequence
H14	The Kafka sink is off the latency path	Hot-path p99 and p99.9, sink on vs off; gaps and duplicates after broker outage	Sink off	Latency difference within noise (threshold TBD); zero gaps and duplicates

9.2 Metrics

Family	Metrics
Discrimination (secondary)	AUC / concordance per horizon
Calibration (primary)	Brier, IBS, ECE, reliability diagrams
Decision (primary)	Net edge per post; markouts at $h \in \{1\,\text{s}, 10\,\text{s}, 60\,\text{s}\}$; across (f, δ, c)
Uncertainty	Coverage, interval width, conditional coverage
Systems	Latency percentiles, throughput, failover time, replica lag
Cost	Inference ns/decision, bytes/model

9.3 The L3 → L2 information-loss study

MBO is ground truth. Degrade it to aggregate price-level data (what most public sources offer), re-infer queue position under three cancel-allocation rules (front, quantity-weighted random, back), and compare against truth. This quantifies what an L2-based backtest of passive execution gets wrong and is motivated by recent work showing the allocation rule alone moves passive completion rates by several percentage points on two Tokyo Stock Exchange instruments [R]. Because that evidence is two instruments over 18 days, the study is repeated here on different products rather than assumed to generalize.

9.4 Ablations

Feature groups; hazard bin grid; model rung; fixed-point vs float features; calibration method; decision threshold c; telemetry on/off; δ grid.

9.5 Statistical protocol and negative results

Confidence intervals come from stationary or day-block bootstrap, never row-wise resampling. The sealed holdout is opened once. Every run writes an immutable manifest (data hash, commit, binary hash, config, seeds, hardware). **Negative or null results are published** in the repository: if M3 does not beat M1 in edge, the report says so, and that is a legitimate result.

---

10. Rust vs C++ Bake-off Protocol

One component is implemented in both languages: **feed decode, sequencing, and journal write**. Both must emit a byte-identical journal for the same input.

Step	Rule
1	Specify the journal format and a conformance test first
2	Implement the same algorithm in both; verify byte equality on the full corpus
3	Fixed time-box (for example two days each) for language-idiomatic optimization
4	Measure throughput and tail latency on the same host, open-loop
5	Record defects found by tooling: ASan/UBSan/TSan for C++, Miri/clippy for Rust
6	Report results, including lines of code and time to first correct implementation

The week-one ingest is written in Rust (a maintained DBN decoding crate exists `[verify before relying]`), with the C++ twin added later. The bake-off extends to the ITCH/MoldUDP64 decoder once it exists, under the same byte-identical-journal rule.

---

11. Phase C: Counterfactual Execution Policy

11.1 A correction to the earlier plan

The initial sketch proposed off-policy evaluation with bandit feedback. That is unnecessary at this stage. Under assumption A1, shadow orders yield counterfactual outcomes for **every passive action at every level simultaneously**, and aggressive actions (crossing the spread for one lot) are likewise computable. This is **full-information feedback**: policy learning reduces to cost-sensitive supervised learning, and "evaluation" is replay under A1. Doubly-robust OPE becomes relevant only when actions have impact or when real logged actions exist.

11.2 Scope

Item	Design
Actions	{post at $\ell \in \{0, 1, 2\}$, cross, wait} per side
State	Features from Section 5.2 plus inventory
Reward	Net markout at chosen horizon, minus inventory penalty
Learning	Cost-sensitive classification / regret-minimizing policy over the model's edge and its lower bounds
Robustness	Evaluate under an impact model (queue-reactive or Hawkes simulator `[R]`) and under queue-jump stress
Claim boundary	No P&L claim. Report edge under stated assumptions with sensitivity bands

11.3 Validating A1 with real fills (optional, later)

The strongest answer to "does a shadow order behave like a real order?" is real small-size orders on a venue that publishes an order-level feed, comparing realized fill times with shadow predictions. This introduces

real-money, venue-terms, and regulatory considerations and is **not** in scope until separately reviewed.

---

12. Verification Strategy

Layer	What	Tooling	Gate
Unit	Book, queue, feature kernel, labeler	GoogleTest or Catch2	CI
Differential	Engine vs naive $O(n)$ reference over randomized discrete-price streams (existing approach)	Existing harness	CI; injected-bug check keeps it honest
Reconciliation	Reconstructed book vs provider-published book after every event	Replay tool	Zero unexplained mismatch
Labeler exact-match	Shadow vs real unmodified orders	Replay tool	100% match
Property-based	Invariants: FIFO within price, conservation of volume, monotone survival	RapidCheck or equivalent	CI
Golden parity	Offline vs online features; Python emulation vs C++ inference	Hash comparison	Bit-exact
Leak canaries	Planted leaks	Python suite	100% flagged
Fuzz	Journal and feed parsers	libFuzzer; `cargo fuzz`	Nightly
Sanitizers	ASan, UBSan, TSan	CMake options (TSan already integrated)	CI
Allocation trap	No `malloc` in replay	Interposer	CI
Chaos	F1–F8	Fault-injection harness	Pre-release
Mutation checks	Inject known bugs; tests must fail	Manual + scripted	Per milestone
Prefix invariance	Random suffix mutation, deletion, truncation	Custom harness	CI
Reference kernel	Independent integer Python feature implementation	numpy	Bit-exact
Protocol conformance	Round-trip, differential vs reference parser, malformed input	Spec test vectors, libFuzzer	CI; nightly fuzz
Risk gate	Must-block tests; property tests	RapidCheck or equivalent	CI
Model checking	TLA+ spec of failover and fencing	TLC	On every spec change
Deterministic simulation	Seeded fault schedules over the replica pair	Custom simulator	CI subset; nightly full
Sink isolation	Broker outage; latency with sink on vs off	Chaos harness	Pre-release
Performance regression	Tail latency against a stored baseline	CI on a pinned host	Fails the build beyond tolerance (TBD)
Build reproducibility	Pinned toolchain container; binary hashes	Container + CI	Same hash across rebuilds

---

13. Data, Licensing, and Cost

Item	Plan	Constraint
Primary data	CME futures MBO from a commercial vendor, front-month, FIFO product	Free starter credit exists `[R]`; **price per product-day must be checked via the vendor's cost endpoint before each pull**
Large-tick preference	Products whose spread is usually one tick make queue position dominant	Final product choice after cost check
ITCH development data	Nasdaq historical TotalView-ITCH files are served from an account-gated server; a free sample was **not** confirmed	Fallback: encode ITCH 5.0 messages from the vendor's normalized Nasdaq dataset; decoder tests run on converted data and are labeled as such

Item	Plan	Constraint
Redistribution	Do not commit raw vendor data	Commit pull scripts, hashes, manifests
Compute	Spot/on-demand instances only while benchmarking; laptop for development	Known on-demand price for the existing benchmark instance type, [M] from author's prior logs
Hosting	Static results and recorded replays; no always-on service	Avoids recurring cost
Broker	Single containerized Kafka-compatible broker for the sink, run locally	No managed service, no recurring cost
Bare metal	One short session for hardware counters	Check current price before launching

Estimated first-week spend is a few tens of dollars [H], to be replaced by actuals recorded in the journal.

---

14. Risk Register

ID	Risk	Likelihood	Impact	Mitigation / trigger
R1	Data volume or price exceeds free credit	Medium	Medium	Shrink day count / product; use free sample for development
R2	A1 (no impact) materially wrong	Medium	High	Sensitivity bands; deep-queue restriction; state limits plainly
R3	Learned models fail to beat the heuristic	Medium	Medium	Publish it; infrastructure and rigor remain the contribution
R4	Scope creep (HA, Rust, Phase C)	High	High	Tiered scope (Section 15); explicit stop-lines
R5	Implementation-experience risk in C++/Rust	Medium	High	Small units, sanitizers, differential tests, clang-tidy/clippy, review checklists
R6	Nondeterminism sneaks in	Medium	High	Integer-only path; fp-contract off; cross-compiler/ISA hash test
R7	Overclaiming	Medium	High	Evidence labels; claims register (Section 17); proof-of-work journal
R8	Venue rule subtleties (priority on modify, implied orders)	Medium	High	Reconciliation gate; exclude levels affected by implied liquidity
R9	Timestamp semantics misunderstood	Medium	High	Knowability test; documented clock policy
R10	No hardware counters on VMs	High	Low	One bare-metal session
R11	Fixed-point features cost accuracy	Medium	Medium	Measured ablation vs float reference
R12	Online head destabilizes	Low	High	Bounded updates, shadow gate, rollback
R13	Protocol scope (ITCH, OUCH, risk gate, SBE) exceeds capacity	High	High	Protocols start after the week-one slice; each has a bounded exit test; SBE deferred
R14	ITCH source data unavailable, or converted data hides real-feed quirks	Medium	Medium	Label results as converted; use specification test vectors; add a real source if one becomes available
R15	Kafka sink read as a hot-path dependency	Low	Medium	Sink reads the journal only; H14 on/off ablation is published

---

15. Roadmap

15.1 Definition of "80% in one week"

"80%" means **80% of the credibility, not 80% of the features**: a vertical slice from raw feed to a calibrated, leakage-tested prediction with a parity-verified C++ inference path. HA, hot-swap, online adaptation, the bake-off, and Phase C are explicitly later.

Week-one exit criteria

ID	Deliverable	Gate
W1-1	Repo bootstrap: journal, signed commits, CI, manifests	CI green
W1-2	Engine upgrade: fixed-point ticks, flat ladder, cancel/modify semantics	Existing differential suite still passes
W1-3	Feed decoder + sequencer + hash-chained journal (Rust)	Conformance test on sample data; chain verifies
W1-4	Book reconstruction vs provider book	Zero unexplained mismatch over N days
W1-5	Shadow labeler	Exact-match on real unmodified orders
W1-6	Feature kernel (streaming-only API) with offline/online parity, prefix-invariance test, and row audit column	Bit-exact; invariance holds on sampled epochs
W1-7	Dataset builder, CPCV, leak canary suite, M0–M2	Canaries 100% flagged
W1-8	First calibrated results, pre-registered	Journal entry before run, artifacts after
W1-9	C++ inference (compiled trees or int8 MLP) with parity	Bit-exact vs Python emulation
W1-10	Minimal telemetry: counters and HDR latency	Informal laptop numbers labeled as such

15.2 Beyond week one

Window	Focus
Weeks 2–4	Int8 MLP (M3); full CPCV; δ-sensitivity (H3); L3→L2 study (H4); ITCH/MoldUDP64 handler with loss injection (H11); PIT1–PIT7 and the reference kernel (H12); risk gate (H13); EC2 benchmarks; telemetry dashboards
Weeks 5–8	Replicas, state hashing, fault injection (F1–F11); OUCH codec, session, and mock-exchange loop; TLA+ failover spec and deterministic simulation; hot-swap and shadow mode; resolve engine throughput reporting
Months 3–4	Rust/C++ bake-off (H10); conformal intervals and delayed-feedback ACI (H6); constrained online head; Kafka sink and outage test (H14); build container and performance-regression CI; postmortem of an injected incident; SBE journal-schema decision
Months 4–6	Phase C policy and robustness; second product or venue; bare-metal profile; per-release leakage report; technical report (LaTeX); public release

Each window ends with a journal entry, a tagged release, and an updated claims register.

---

16. Repository Layout and Reproducibility

```
fillcast/
├── docs/
│   ├── DESIGN.md            # this document
│   └── journal.md           # dated decisions, experiments, failures
├── engine/                  # C++20 data plane (from the order book engine)
├── ingest/                  # Rust control plane: decode, sequence, journal
├── protocols/               # ITCH/MoldUDP64 and OUCH codecs; spec test vectors
├── gateway/                 # risk gate, order session, mock exchange
├── sinks/                   # Kafka sink sidecar (off the latency path)
├── formal/                  # TLA+ specs, simulation seeds
├── research/                # Python: dataset builder, CPCV, models, reports
├── specs/                   # journal format, model blob format, conformance tests
├── manifests/               # data hashes, pull scripts (no raw data)
├── results/                 # dated, hardware-named captures; only source of numbers
└── Makefile                 # make reproduce
```

Protocol. Every reported number comes from a captured run in `results/` with the hardware named. Commits are small and signed. Hypotheses are logged before experiments, outcomes after, including dead ends.

How this was built. The design was drafted with assistance from an AI coding assistant; architecture decisions, experiments, and code are reviewed and verified by the author. This section is kept accurate as the project evolves.

---

17. Claims Register

Claims enter the README and resume only when they have an artifact. This table starts empty by design.

Claim	Evidence artifact	Hardware	Status
(none yet)			

Relation to the existing engine: the engine is the substrate of the hot path. Its published benchmark figures are a separate claims set with its own open items (the throughput table was measured under a single-core pinning configuration, and a dual-core run measured higher). Those figures are reconciled **before** both projects are cited together.

---

18. References

Items marked [read] were read in this design session. Others are standard references cited from the author's knowledge and **must be verified against the originals before publication.**

- [read] Berti, L. and Kasneci, G. *TLOB: A Novel Transformer Model with Dual Attention for Price Trend Prediction with Limit Order Book Data.* arXiv:2502.15757.
- [read] Bodor, A. and Carlier, L. *Deep Learning Meets Queue-Reactive: A Framework for Realistic Limit Order Book Simulation.* arXiv:2501.08822.
- [read] *KANFormer for Predicting Fill Probabilities via Survival Analysis in Limit Order Books.* arXiv:2512.05734.
- [read] *Bridging the Reality Gap in Limit Order Book Simulation.* arXiv:2603.24137.
- [read] *Same Book, Different Fills: Partial Identification of FIFO Execution from Aggregate Order Books.* arXiv:2609.13597.
- [read] Xu, C. and Xie, Y. *Sequential Predictive Conformal Inference for Time Series.* arXiv:2212.03463.
- Cont, R., Kukanov, A., and Stoikov, S. *The Price Impact of Order Book Events.* Journal of Financial Econometrics, 2014.
- Cont, R. and de Larrard, A. *Price Dynamics in a Markovian Limit Order Market.* SIAM J. Financial Mathematics, 2013.
- Huang, W., Lehalle, C.-A., and Rosenbaum, M. *Simulating and Analyzing Order Book Data: The Queue-Reactive Model.* JASA, 2015.
- Gibbs, I. and Candès, E. *Adaptive Conformal Inference Under Distribution Shift.* NeurIPS, 2021.
- Vovk, V. and Petej, I. *Venn–Abers Predictors.* UAI, 2014.
- López de Prado, M. *Advances in Financial Machine Learning.* Wiley, 2018 (purging, embargo, CPCV).
- Bailey, D. and López de Prado, M. *The Deflated Sharpe Ratio.* Journal of Portfolio Management, 2014.
- Bailey, D., Borwein, J., López de Prado, M., and Zhu, Q. *The Probability of Backtest Overfitting.* Journal of Computational Finance, 2017.
- Tsiatis, A. *A Nonidentifiability Aspect of the Problem of Competing Risks.* PNAS, 1975.
- Tene, G. *How NOT to Measure Latency* (coordinated omission), conference talk.
- Nasdaq. *TotalView-ITCH 5.0* and *MoldUDP64* specifications (nasdaqtrader.com). Surfaced by search; not read in full.
- Nasdaq. *OUCH* order-entry and *SoupBinTCP* specifications. Not yet read.
- Lamport, L. *Specifying Systems.* Addison-Wesley, 2002 (TLA+).
- Wilson, W. *Testing Distributed Systems w/ Deterministic Simulation.* Strange Loop, 2014 (the FoundationDB approach).
- U.S. Securities and Exchange Commission. *Rule 15c3-5, Risk Management Controls for Brokers or Dealers with Market Access.* Cited for the nature of pre-trade controls only; no compliance claim.

Appendix A: Glossary

Term	Meaning
MBO / L3	Market-by-order: every order add, cancel, modify, and trade
L2	Aggregate size per price level
Shadow order	Hypothetical order tracked by replay without altering the book
Knowability clock	Rule that features use only data available at decision time
CPCV	Combinatorial purged cross-validation
Markout	Signed mid-price change after a fill, relative to fill price
Adverse selection	Tendency for passive fills to precede unfavorable price moves
Coordinated omission	Latency under-measurement when a load generator waits on slow responses
RCU	Read-copy-update: swap a shared pointer, reclaim the old object after readers finish
ITCH	Nasdaq's order-by-order market data protocol
MoldUDP64	UDP transport for ITCH with sequence numbers and retransmission requests
OUCH / SoupBinTCP	Nasdaq's order-entry protocol and the TCP session layer it runs over
SBE	Simple Binary Encoding, a schema-driven binary message format
Prefix invariance	Property that features at earlier events are unchanged by any change to later events
Bitemporal	Records that carry both when a fact was valid and when it was recorded
Deterministic simulation testing	Running a system in a simulated environment under seeded faults so any failure replays exactly
TLA+ / TLC	A specification language for concurrent systems and its model checker

Appendix B: Building the Technical Report

The Markdown source is Pandoc-compatible: YAML metadata block, pipe tables, `$...$` math, and fenced code with language tags. Diagrams are plain-text for portability and are redrawn in TikZ for the final PDF.

```
pandoc docs/DESIGN.md \
  --from markdown+tex_math_dollars+pipe_tables \
  --pdf-engine=xelatex \
  --toc --number-sections \
  -V geometry:margin=1in -V colorlinks=true \
  -o build/fillcast-design.pdf
```